CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação
Prof. Marco André Abud Kappel

Aula 2 – Ferramentas e ambiente

Ferramentas e ambiente

- **Por que Python?**
  - Python se tornou a **linguagem padrão** para a maioria das aplicações de Data Science.
  - Ela combina o poder de uma linguagem de **propósito geral** com a facilidade de uso de linguagens específicas para **uso científico**.

Ferramentas e ambiente

- **Por que Python?**
  - Isso é possível porque Python possui muitas **bibliotecas** excelentes para carregar, manipular, visualizar, analisar e calcular estatísticas sobre dados.
  - Além disso, ele possui bibliotecas específicas para processamento de imagens, processamento de linguagem natural e, é claro, métodos de **Machine Learning**.

Ferramentas e ambiente

- **Por que Python?**
  - Uma das principais vantagens do Python é a habilidade de interagir diretamente com o código, usando um terminal.
  - A criação de um **script** de aprendizado de máquina envolve a **modificação frequente do código** e análises frequentes dos **dados em memória**.
  - Assim, é essencial que tenhamos ferramentas que possibilitem uma fácil e rápida **interação com os dados**.
  - Além disso, o Python permite a criação de GUIs complexas e web services, para integração com outros sistemas.

Ferramentas e ambiente

- **Anaconda**

ANACONDA®

  - Anaconda é uma **plataforma de desenvolvimento** de Python voltada para a área de Data Science.
  - A Distribuição Anaconda já acompanha as principais ferramentas de processamento de dados, análise preditiva e computação científica do Python:

    - **Scikit-learn**
    - **NumPy**
    - SciPy
    - **Matplotlib**
    - **Spyder**
    - **Pandas**
    - Ipython
    - Jupyter Notebook

Ferramentas e ambiente

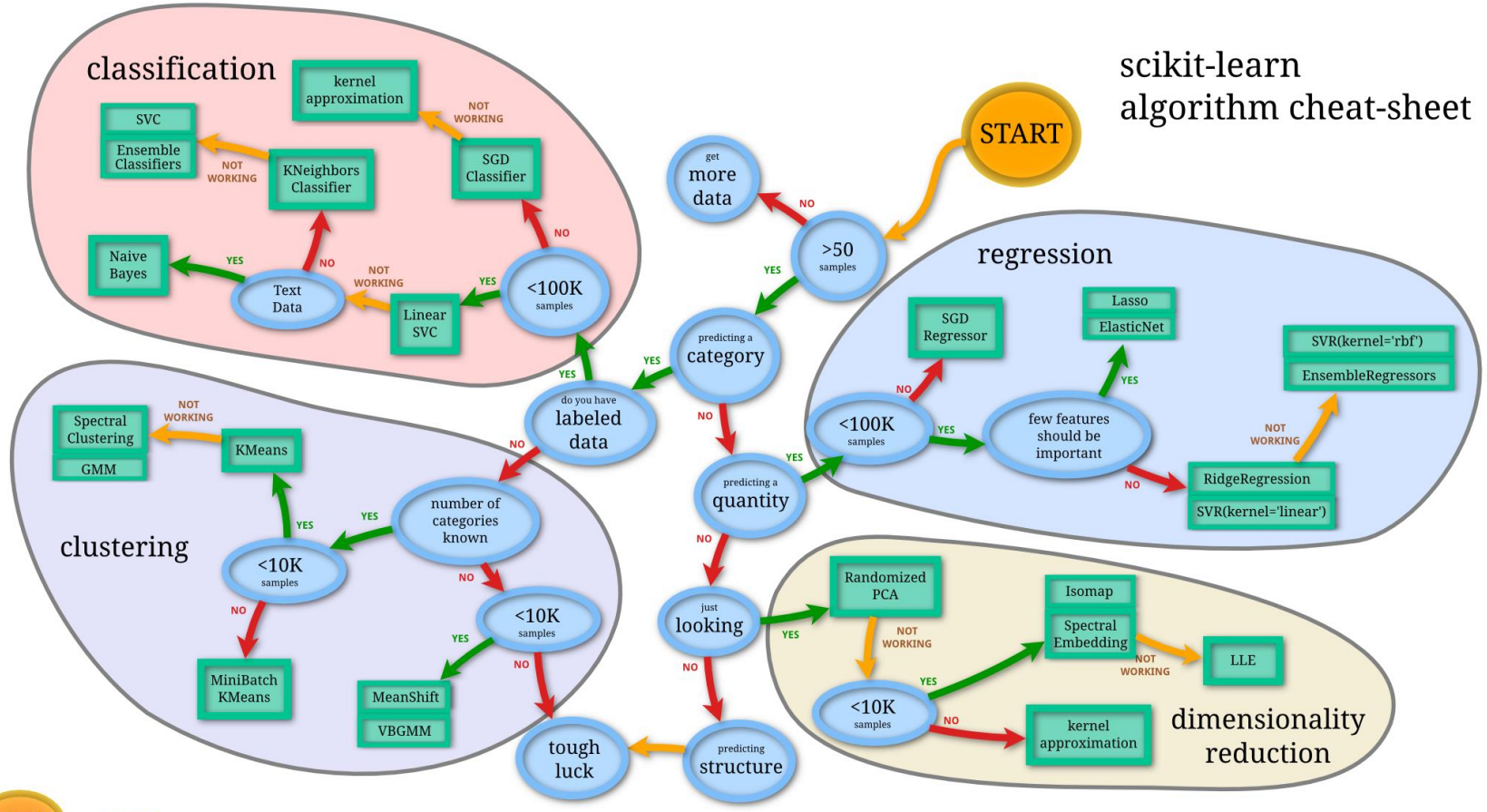
- **Anaconda**
  - Para instalar basta entrar em: https://www.anaconda.com/download/

Ferramentas e ambiente

- **Scikit-learn**

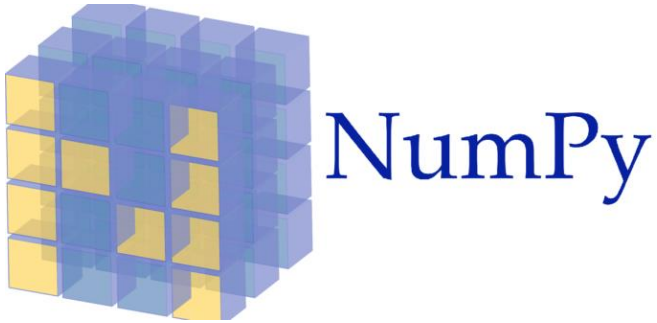
  - É um projeto open-source (gratuito para usar e distribuir) com uma **comunidade muito ativa**.
  - Ele contém a implementação de um grande número de **algoritmos** de Machine Learning do **estado da arte**.
  - Também possui uma extensa e detalhada **documentação**.
  - Por esses motivos, é uma das **ferramentas** mais utilizadas na indústria e no meio acadêmico.

Ferramentas e ambiente



Ferramentas e ambiente

- **Numpy**

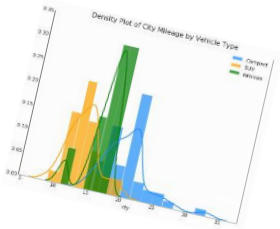


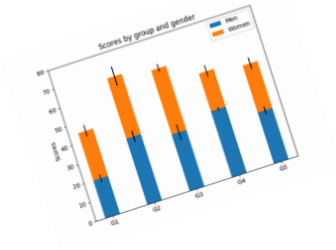
  - Um dos pacotes fundamentais para **computação científica** com Python.
  - Contém **funcionalidades** para arrays multidimensionais, funções matemáticas de alto nível, como operações de álgebra linear e transformada de Fourier, e gerador de números pseudorrandômicos.
  - O **array do numpy** é a estrutura de dados fundamental do scikit-learn.

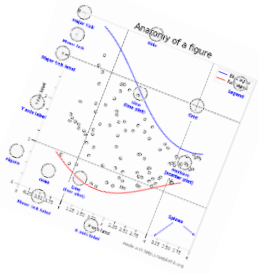
Ferramentas e ambiente

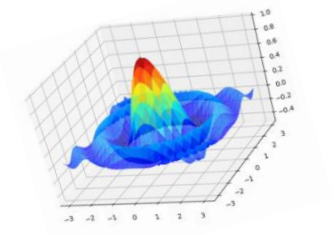
- **Matplotlib**

  - É a principal biblioteca científica para **plotar gráficos** em Python.
  - Fornece funções para **visualização** em alta qualidade de gráficos de linha, dispersão, histogramas, etc.
  - Visualizar seus dados de forma simples e rápida pode possibilitar análises importantes.







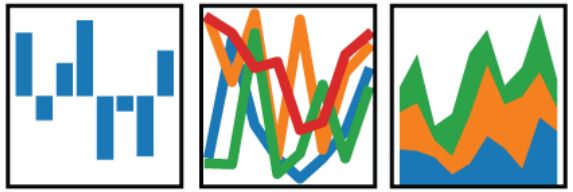


Ferramentas e ambiente

- **Pandas**

pandas

$y_i t = \beta' x_{it} + \mu_i + \epsilon_{it}$



  - É uma biblioteca de Python para **análise e comparação** de dados.
  - Utiliza uma estrutura de dados chamada **DataFrame**, uma espécie de tabela de fácil visualização.
  - Permite que os valores da tabela sejam de tipos diferentes e fornece diversas maneiras de interagir com seus dados.
  - Sobretudo, ela permite a “ingestão” de uma grande variedade de bases de dados (arquivos CSV, planilha Excel, SQL, entre outros).

Ferramentas e ambiente

- **Spyder**

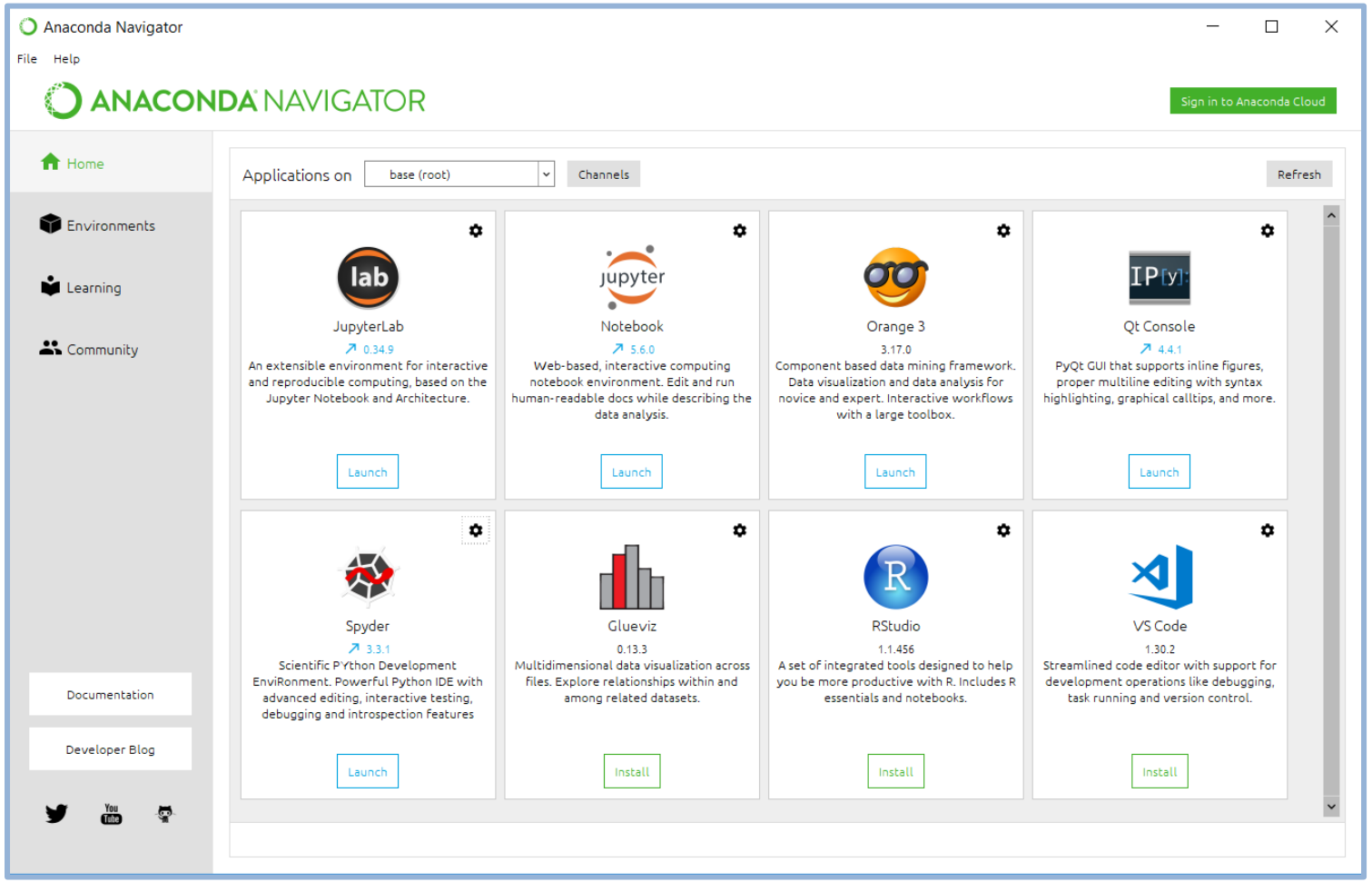
SPYDER

The Scientific Python Development Environment

  - É a **IDE** que utilizaremos para desenvolver nossos scripts.
  - O Spyder é um poderoso **ambiente** científico escrito em Python, para Python, projetado por e para cientistas, engenheiros e analistas de dados.
  - Ele oferece uma combinação de **recursos avançados** de edição, análise, depuração e uma ferramenta de desenvolvimento abrangente com exploração de dados, execução interativa, inspeção detalhada e recursos de visualização de um pacote científico.

Ferramentas e ambiente

- **Spyder**



Ferramentas e ambiente

- **Spyder**

Spyder (Python 3.7)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

O:\Projetos\Aprendizado Supervisionado\Classificação\KNN (base census)

Editor - E:\Dr. Kappel\Cursos e Especialização_Machine Learning e Data Science com Python de A à Z - COMPRADO\Pr...

knn-census.py

```python
import pandas as pd

base = pd.read_csv('census.csv')

previsores = base.iloc[:, 0:14].values
classe = base.iloc[:, 14].values

################## #Pré-processamento dos dados ##################

from sklearn.preprocessing import LabelEncoder, OneHotEncoder
labelencoder_previsores = LabelEncoder()
previsores[:, 1] = labelencoder_previsores.fit_transform(previsores[:, 1])
previsores[:, 3] = labelencoder_previsores.fit_transform(previsores[:, 3])
previsores[:, 5] = labelencoder_previsores.fit_transform(previsores[:, 5])
previsores[:, 6] = labelencoder_previsores.fit_transform(previsores[:, 6])
previsores[:, 7] = labelencoder_previsores.fit_transform(previsores[:, 7])
previsores[:, 8] = labelencoder_previsores.fit_transform(previsores[:, 8])
previsores[:, 9] = labelencoder_previsores.fit_transform(previsores[:, 9])
previsores[:, 13] = labelencoder_previsores.fit_transform(previsores[:, 13

#onehotencoder = OneHotEncoder(categorical_features = [1,3,5,6,7,8,9,13])
onehotencoder = OneHotEncoder(categories='auto')
previsores = onehotencoder.fit_transform(previsores).toarray()

labelencoder_classe = LabelEncoder()
classe = labelencoder_classe.fit_transform(classe)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
previsores = scaler.fit_transform(previsores)

from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, classe_treinamento, classe_teste

#####################################################################

################## Classificação com Naive Bayes ##################
```

Explorador de variáveis

Nome	Tipo	Tamanho	Valor
base	DataFrame	(32561, 15)	Column names: age, w…
classe	object	(32561,)	ndarray object of numpy module
classe_teste	object	(4885,)	ndarray object of numpy module
classe_treinamento	object	(27676,)	ndarray object of numpy module
previsores	object	(32561, 14)	ndarray object of numpy module
previsores_teste	object	(4885, 14)	ndarray object of numpy module
previsores_treinamento	object	(27676, 14)	ndarray object of numpy module

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 2/A

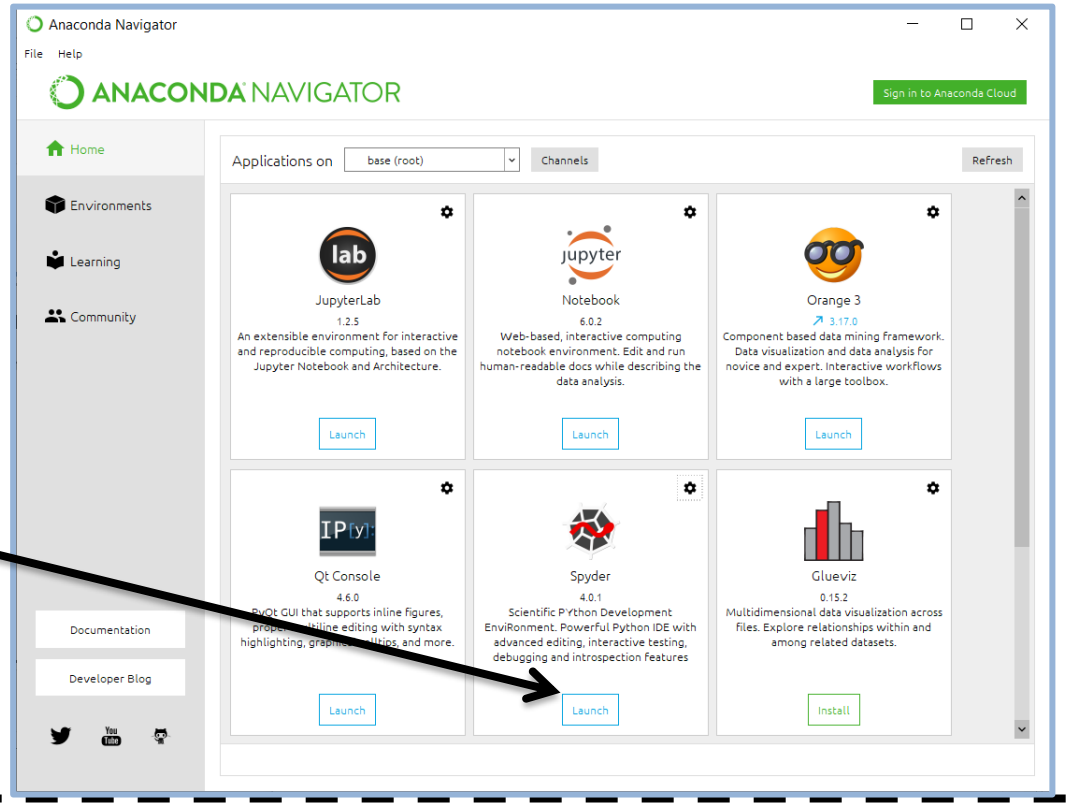
```
   ...:
   ...: base = pd.read_csv('census.csv')
   ...:
   ...: previsores = base.iloc[:, 0:14].values
   ...: classe = base.iloc[:, 14].values
   ...:
   ...: ################## #Pré-processamento dos dados
##################
   ...:
   ...: from sklearn.preprocessing import LabelEncoder,
OneHotEncoder
   ...: labelencoder_previsores = LabelEncoder()
   ...: previsores[:, 1] =
labelencoder_previsores.fit_transform(previsores[:, 1])
   ...: previsores[:, 3] =
labelencoder_previsores.fit_transform(previsores[:, 3])
```

Console IPython | Log do histórico

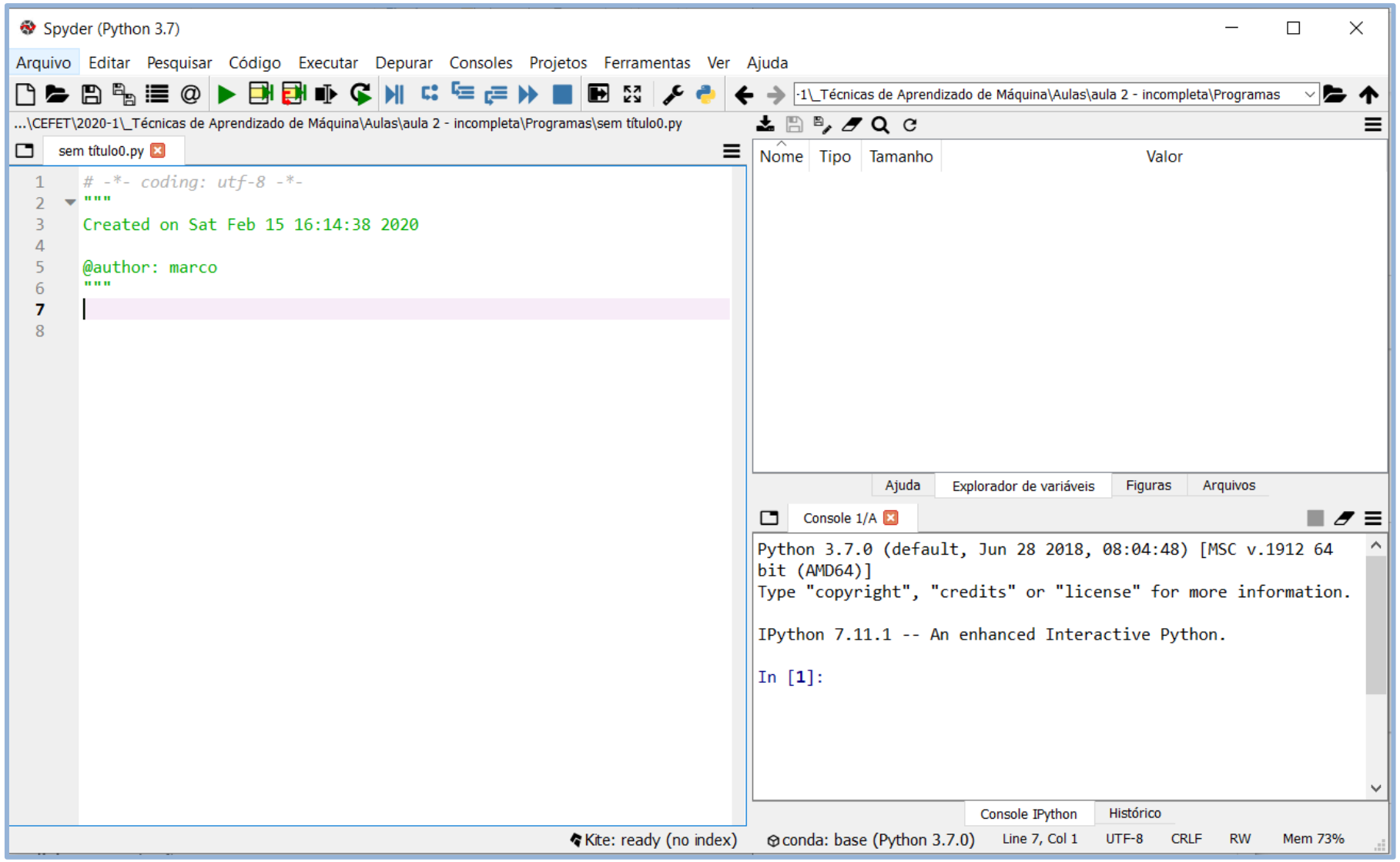
Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8-GUESSED | Linha: 43 | Coluna: 62 | Memória: 28 %

Ferramentas e ambiente

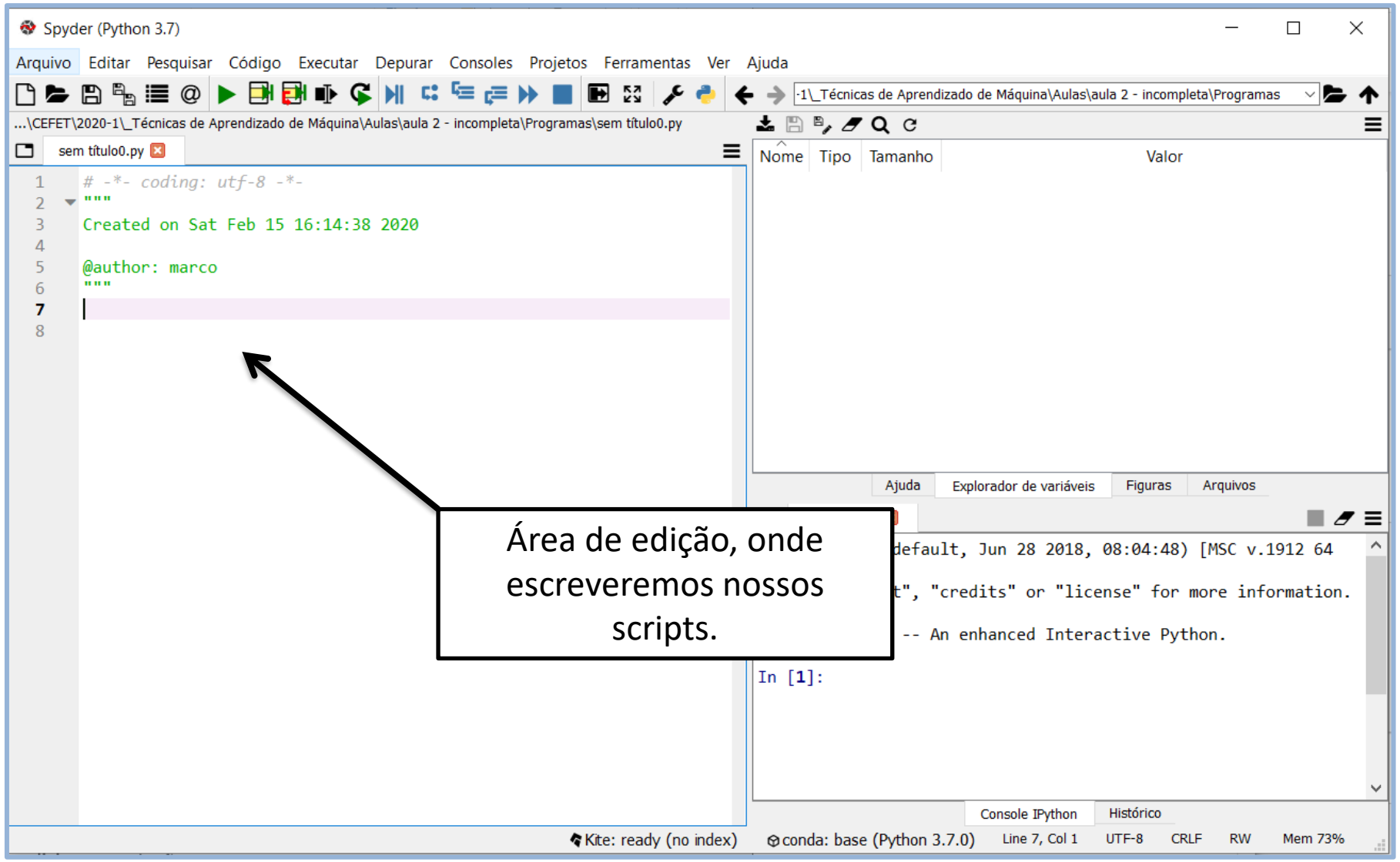
- **Abrindo a IDE**
  - Dentro do Anaconda, abra o Spyder:



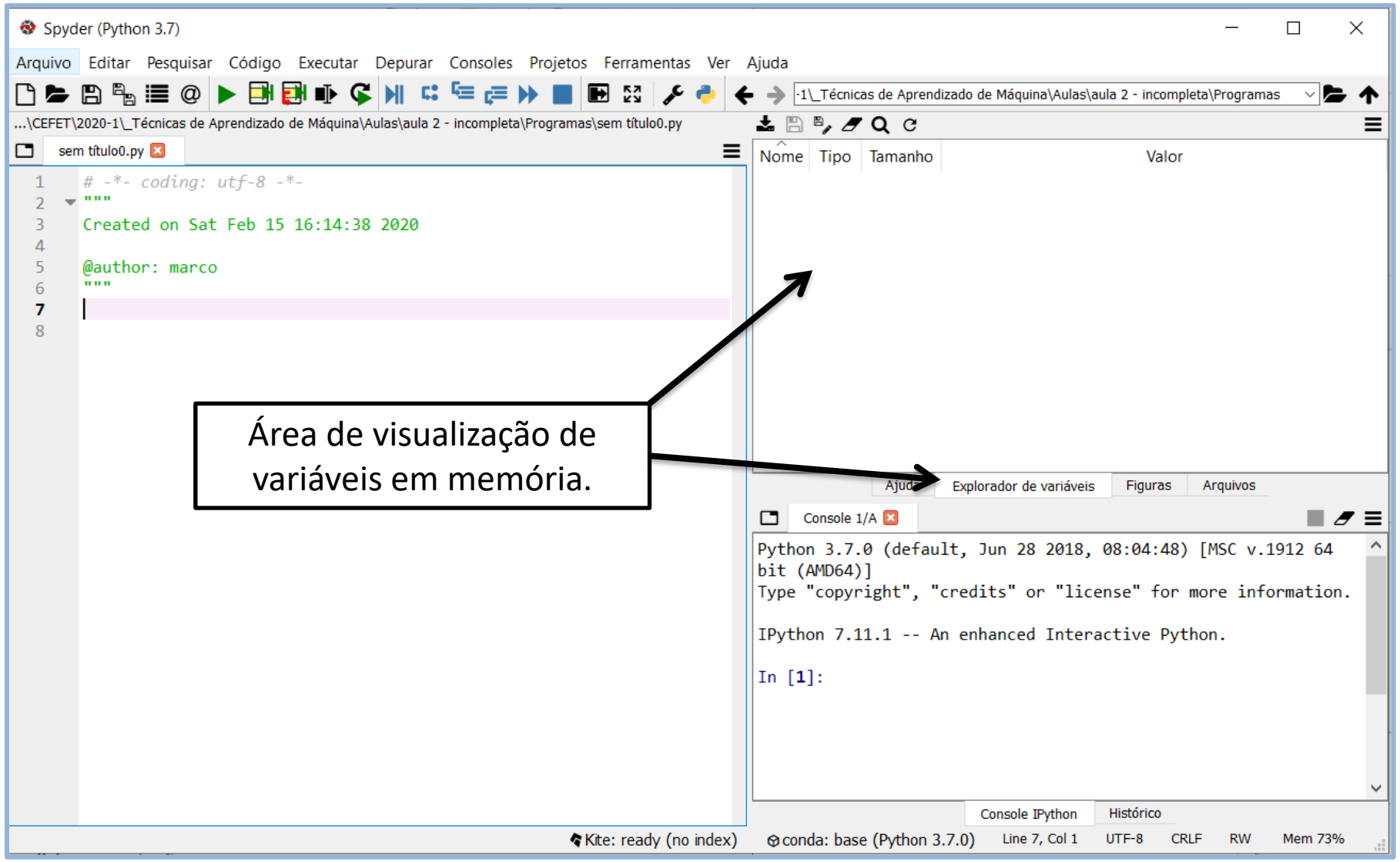
- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.



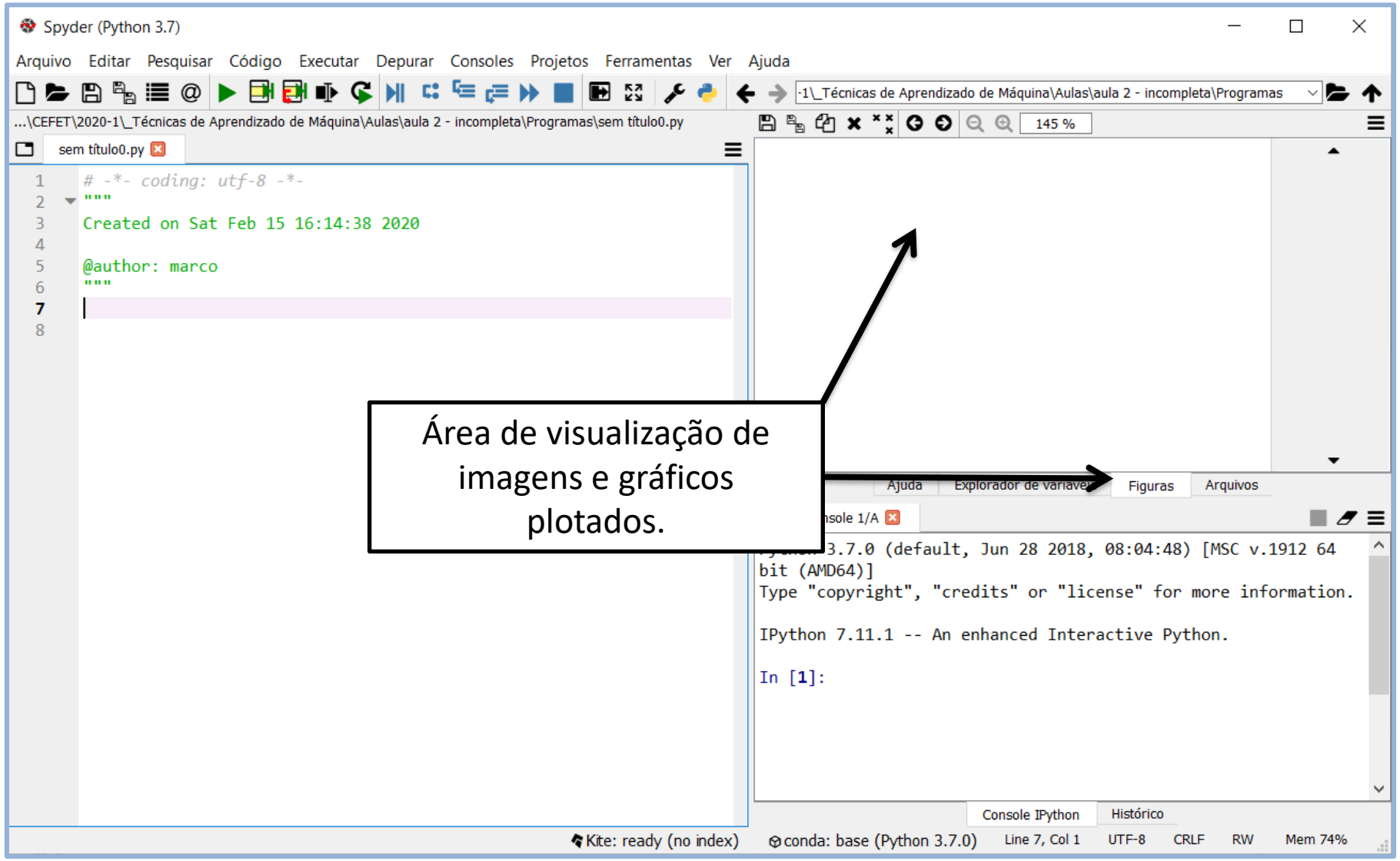
- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.



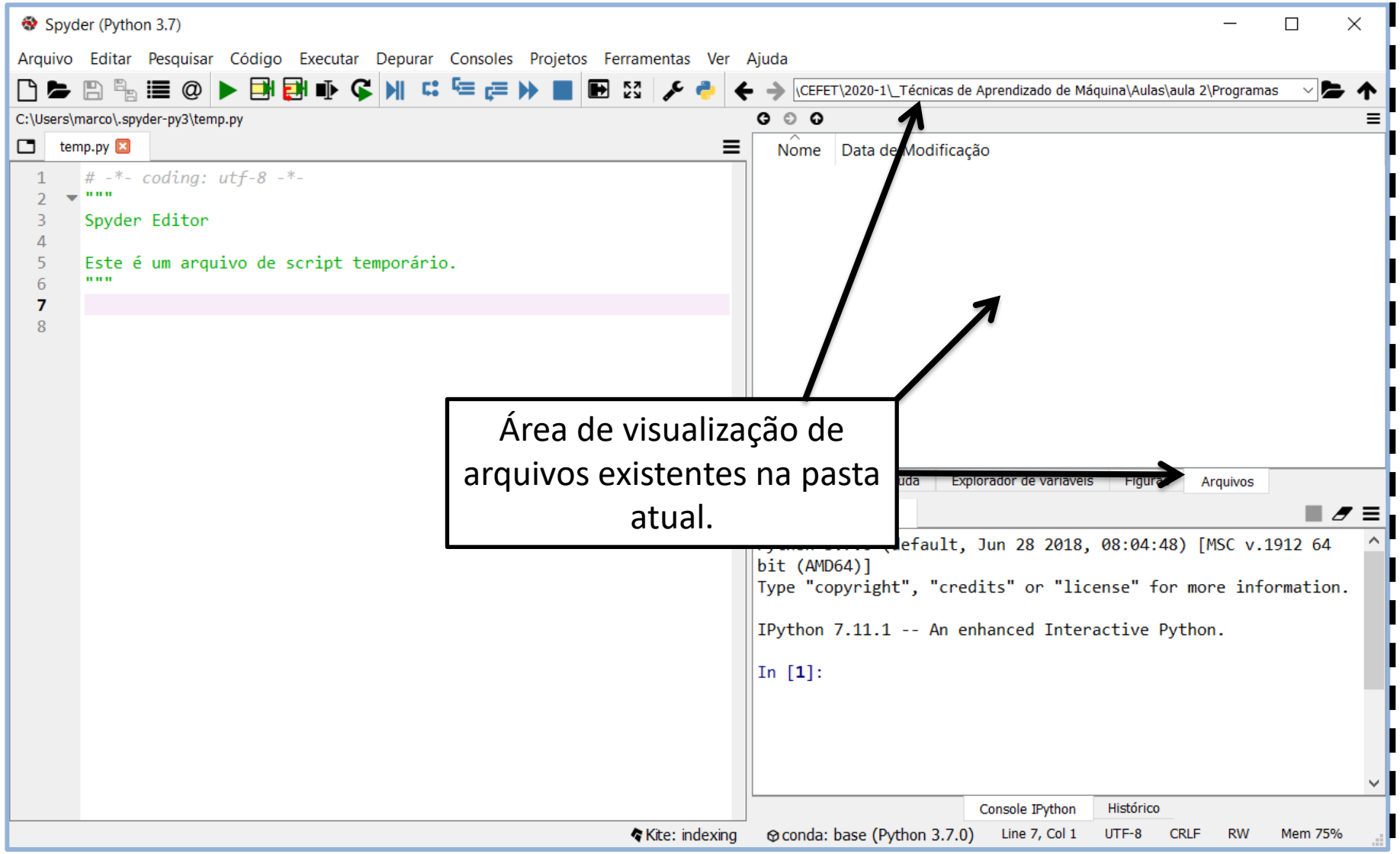
- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.



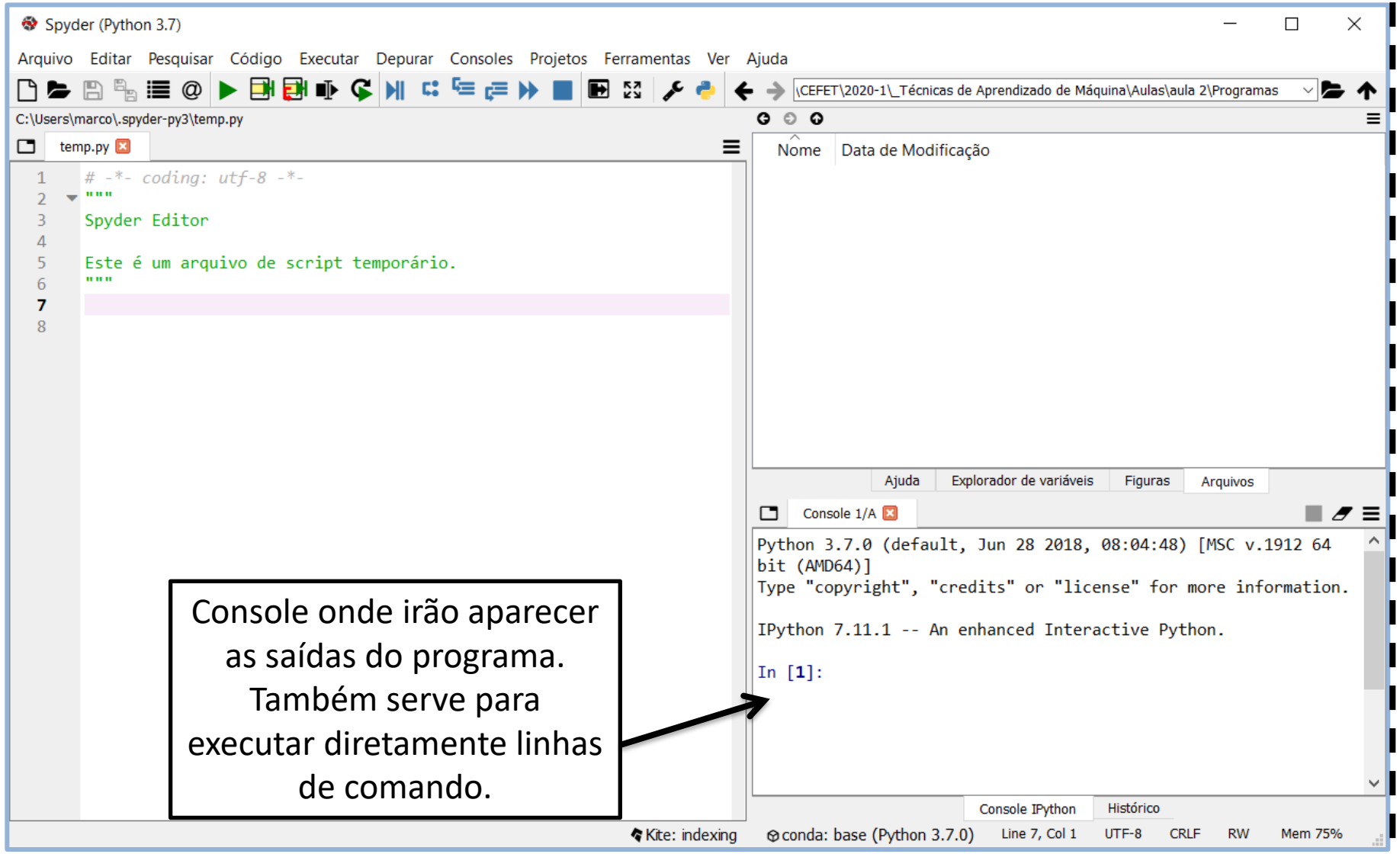
- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.



- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.

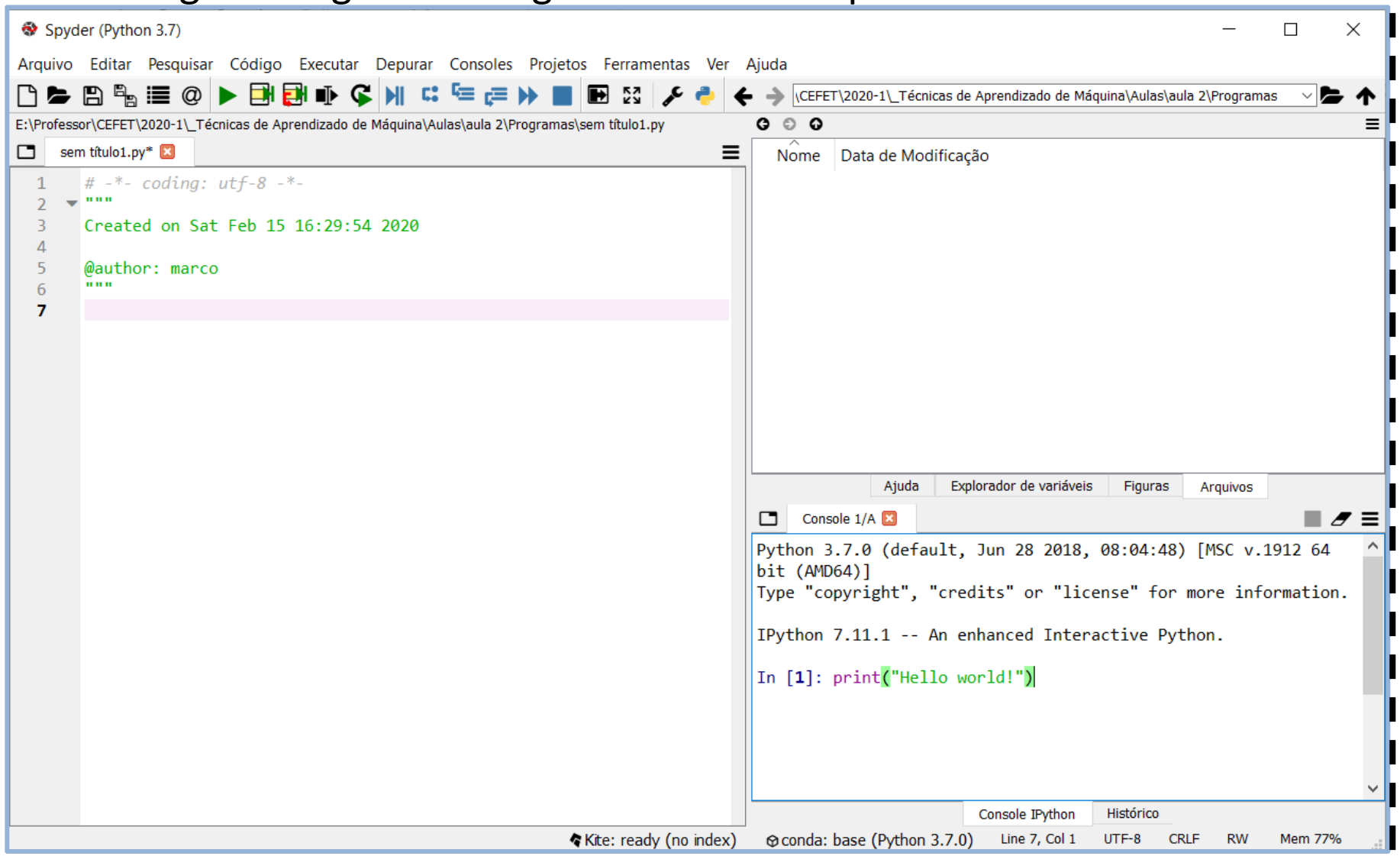


- **Ambiente de programação**
  - Este será o ambiente em que trabalharemos.



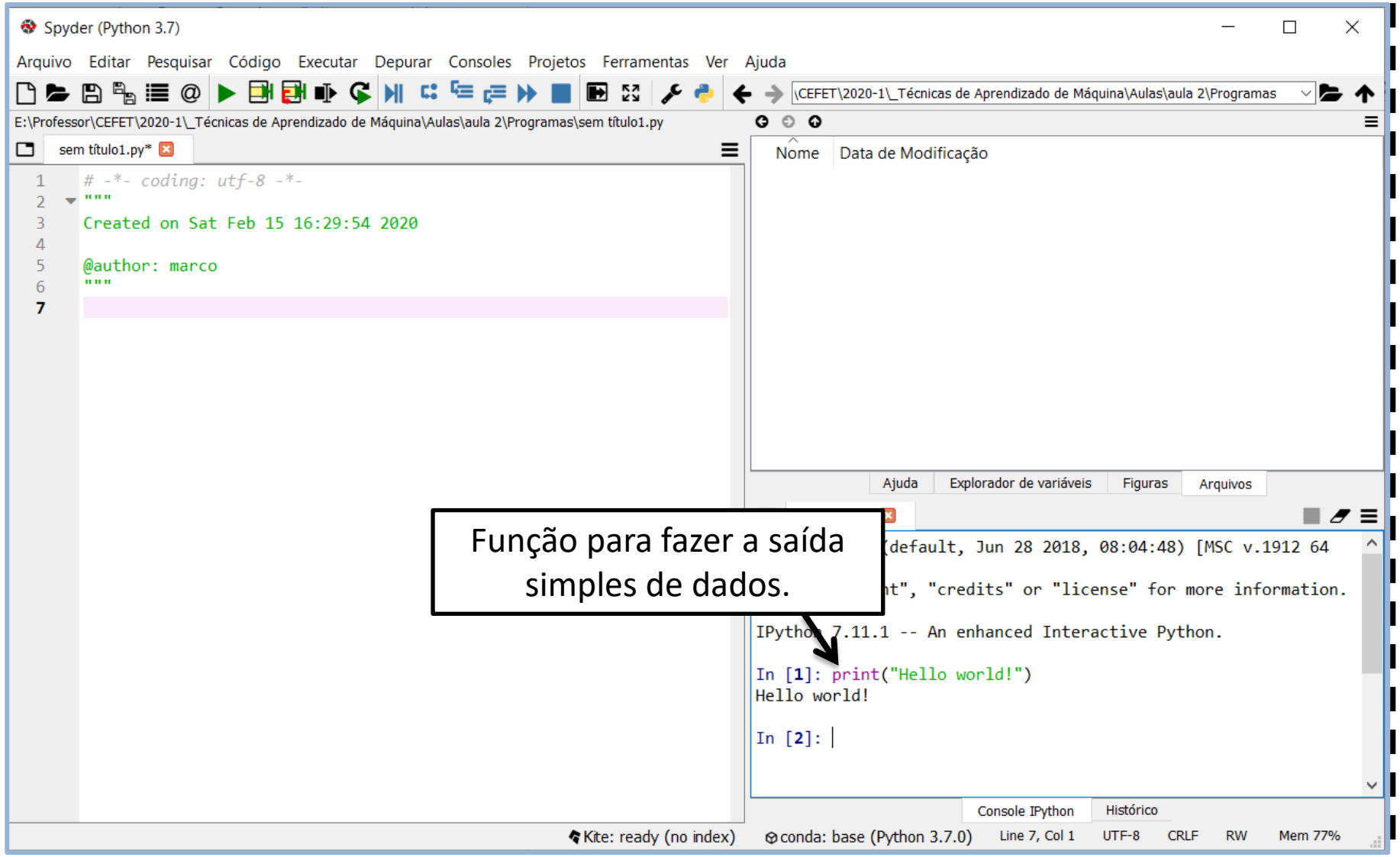
Console onde irão aparecer as saídas do programa. Também serve para executar diretamente linhas de comando.

- **Primeiro teste**
  - Digite o seguinte código no console e aperte **enter**:



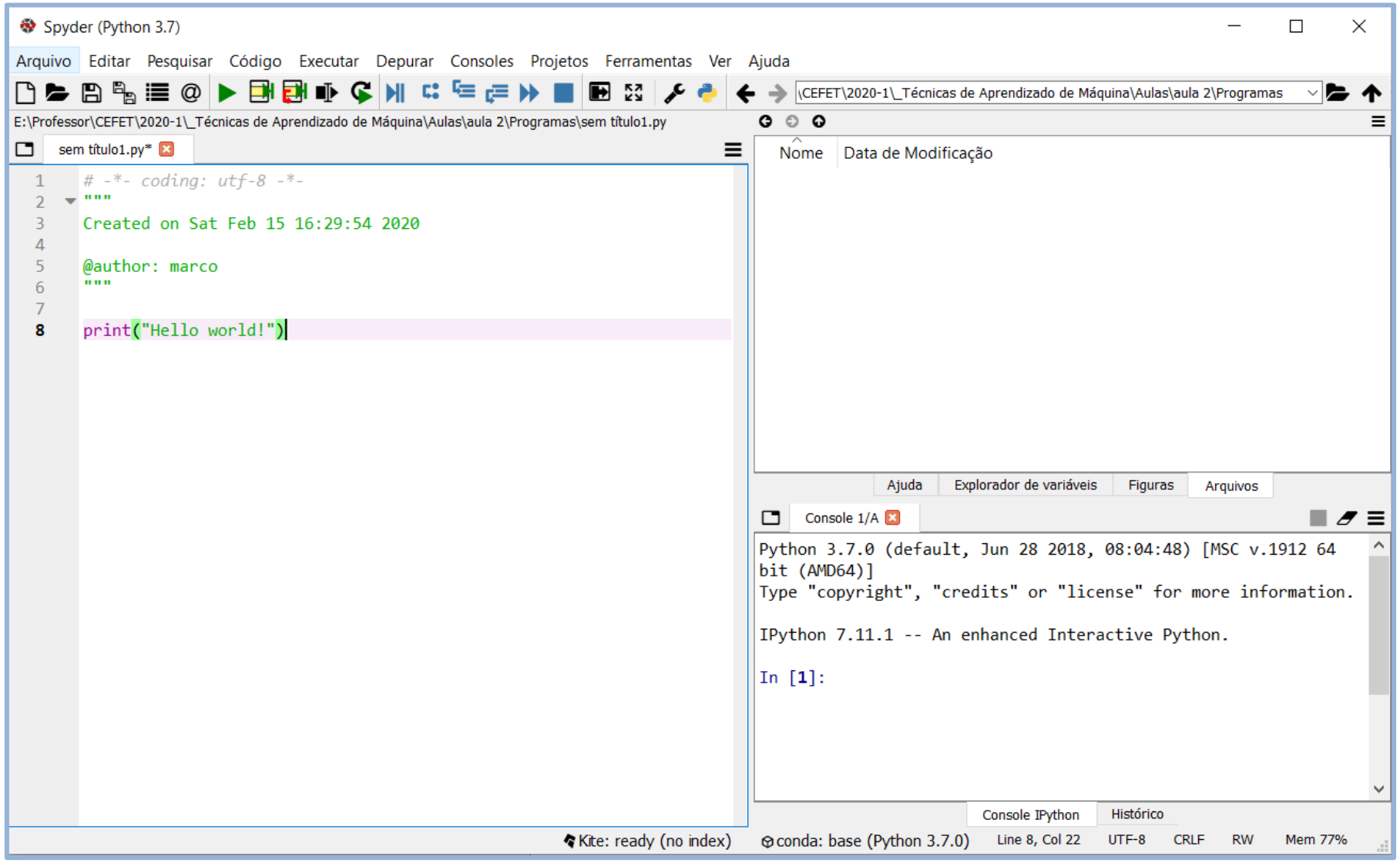
- **Primeiro teste**

  – Digite o seguinte código no console e aperte **enter**:



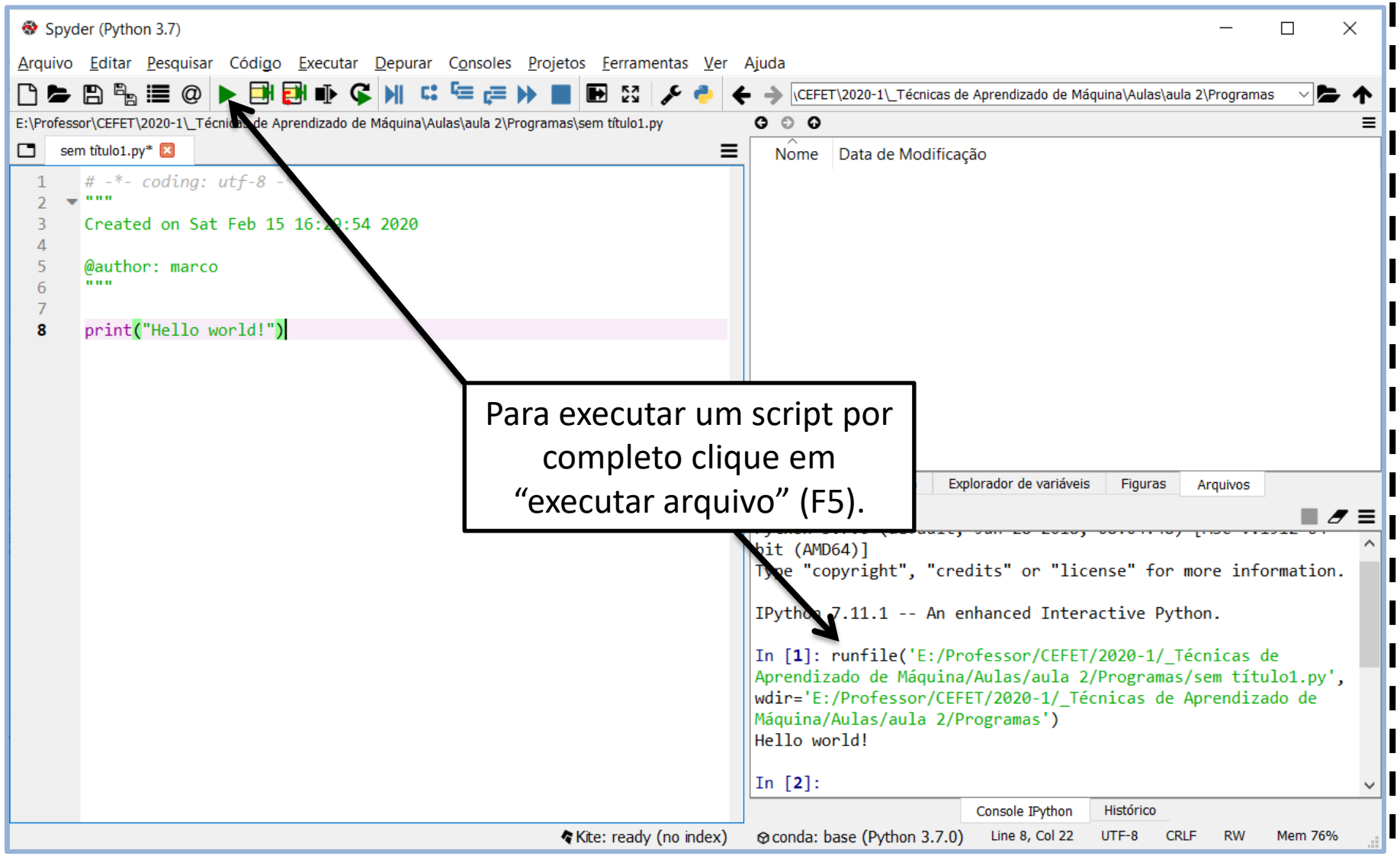
- **Primeiro script**

  – Crie um **novo arquivo** e escreva o seguinte código:



- **Primeiro script**

  – Crie um **novo arquivo** e escreva o seguinte código:



Para executar um script por completo clique em "executar arquivo" (F5).

- **Primeiro script**

  – Muitas vezes, precisamos executar apenas **parte** do código.

Spyder (Python 3.7)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

E:\Professor\CEFET\2020-1_Técnicas de Aprendizado de Máquina\Aulas\aula 2\Programas\sem título1.py

sem título1.py*

```
# -*- coding: utf-8 -*-
"""
Created on Sat Feb 15 16:29:54 2020

@author: marco
"""

print("Hello world!")
```

Para executar apenas um trecho do script, selecione as linhas desejadas e clique em “executar seleção” (F9).

```
An enhanced Interactive Python.

In [1]: runfile('E:/Professor/CEFET/2020-1/_Técnicas de Aprendizado de Máquina/Aulas/aula 2/Programas/sem título1.py', wdir='E:/Professor/CEFET/2020-1/_Técnicas de Aprendizado de Máquina/Aulas/aula 2/Programas')
Hello world!

In [2]: print("Hello world!")
Hello world!

In [3]:
```

Kite: ready (no index) | conda: base (Python 3.7.0) | Line 8, Col 22 | UTF-8 | CRLF | RW | Mem 77%

Ferramentas e ambiente

- **Linguagem Python**
  - Todas as **implementações** que faremos nesta disciplina serão feitas utilizando a linguagem Python.
  - Assim como qualquer outra linguagem, o Python possui sua própria **sintaxe** e **particularidades**.
  - Veremos agora os **comandos básicos** e a sintaxe da programação em Python.

Ferramentas e ambiente

- **Comentários**
  - Você pode fazer comentários de **uma linha** ou de **múltiplas linhas**.

```
# Comentário de uma linha

"""
Comentário de múltiplas linhas
Segunda linha
"""
```

O atalho para comentar uma linha é CTRL+1.

Ferramentas e ambiente

- **Variáveis**
  - Os **identificadores** devem começar com letras ou "_".
  - Não é necessário declarar o tipo da variável, Python é uma linguagem **dinamicamente** tipada.
  - Na verdade, os nomes das variáveis são rótulos de objetos.
  - Em Python, tudo são objetos.

```
meuInteiro = 1
meuFloat = 3.14
minhaString = "Hello World"
meuLogico = True
meuComplexo = 2+3j
```

- **Variáveis**
  - Todas as variáveis criadas irão para o explorador de variáveis.

Spyder (Python 3.7)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramentas Ver Ajuda

\CEFET\2020-1_Técnicas de Aprendizado de Máquina\Aulas\aula 2\Programas

E:\Professor\CEFET\2020-1_Técnicas de Aprendizado de Máquina\Aulas\aula 2\Programas\exemplo.py

exemplo.py

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Sat Feb 15 16:29:54 2020
4
5  @author: marco
6  """
7
8  print("Hello world!")
9
10 meuInteiro = 1
11 meuFloat = 3.14
12 minhaString = "Hello World"
13 meuLogico = True
14 meuComplexo = 2+3j
```

Nome	Tipo	Tamanho	Valor
meuComplexo	complex	1	(2+3j)
meuFloat	float	1	3.14
meuInteiro	int	1	1
meuLogico	bool	1	True
minhaString	str	1	Hello World

Ajuda | Explorador de variáveis | Figuras | Arquivos

Console 1/A

```
Hello world!

In [2]: print("Hello world!")
Hello world!

In [3]: meuInteiro = 1
   ...: meuFloat = 3.14
   ...: minhaString = "Hello World"
   ...: meuLogico = True
   ...: meuComplexo = 2+3j

In [4]:
```

Console IPython | Histórico

Kite: indexing | conda: base (Python 3.7.0) | Line 10, Col 1 | UTF-8 | CRLF | RW | Mem 81%

Ferramentas e ambiente

- **Listas**
  - Exemplo:

```
a = 'muito'
b = 'boa'
umaLista = ['uma', 'lista', a, b]
outraLista = [[4,5,6,7],[3,4,5,6]]
c = umaLista[1]
d = umaLista[-2]
e = umaLista[1:3]
f = umaLista[1:]
g = umaLista[:3]
h = umaLista[:]
i = outraLista[1][0
j = outraLista[1][:
```

Ferramentas e ambiente

- **Listas**
  - Exemplo:

```
a = 'muito'
b = 'boa'
umaLista = ['uma', 'lista', a, b]
outraLista = [[4,5,6,7],[3,4,5,6]]
c = umaLista[1]      # retorna elemento 1
d = umaLista[-2]     # retorna penúltimo elemento
e = umaLista[1:3]    # retorna elementos de 1 a 2
f = umaLista[1:]     # retorna elementos a partir do indice 1
g = umaLista[:3]     # retorna elementos anteriores ao indice 3
h = umaLista[:]      # retorna todos os elementos
i = outraLista[1][0]  # retorna elemento 0 da sublista 1
j = outraLista[1][:2] # retorna elementos anteriores ao indice 2 da sublista 1
```

Nome	Tipo	Tamanho	Valor
a	str	1	muito
b	str	1	boa
c	str	1	lista
d	str	1	muito
e	list	2	['lista', 'muito']
f	list	3	['lista', 'muito', 'boa']
g	list	3	['uma', 'lista', 'muito']
h	list	4	['uma', 'lista', 'muito', 'boa']
i	int	1	3
j	list	2	[3, 4]
outraLista	list	2	[[4, 5, 6, 7], [3, 4, 5, 6]]
umaLista	list	4	['uma', 'lista', 'muito', 'boa']

Ferramentas e ambiente

- **Listas**
  - Exemplo:

```
indice = umaLista.index(a)
conta = umaLista.count(a)
umaLista.append('!')
umaLista.remove('!')
del(umaLista[0:1])
umaLista.reverse()
umaLista.extend('!')
umaLista.pop(-1)
umaLista.insert(0,'!')
umaLista.sort()
```

Ferramentas e ambiente

- **Listas**
  - Exemplo:

```
indice = umaLista.index(a)
conta = umaLista.count(a)
umaLista.append('!')
umaLista.remove('!')
del(umaLista[0:1])
umaLista.reverse()
umaLista.extend('!')
umaLista.pop(-1)
umaLista.insert(0,'!')
umaLista.sort()
```

Nome	Tipo	Tamanho	Valor
a	str	1	muito
b	str	1	boa
c	str	1	lista
conta	int	1	1
d	str	1	muito
e	list	2	['lista', 'muito']
f	list	3	['lista', 'muito', 'boa']
g	list	3	['uma', 'lista', 'muito']
h	list	4	['uma', 'lista', 'muito', 'boa']
i	int	1	3
indice	int	1	2
j	list	2	[3, 4]
outraLista	list	2	[[4, 5, 6, 7], [3, 4, 5, 6]]
umaLista	list	4	['!', 'boa', 'lista', 'muito']

Ferramentas e ambiente

- **Dicionários**
  - Outra estrutura bastante utilizada em Python são os **dicionários**.
  - Nele, você deve especificar um conjunto de **chaves** e os seus **valores**:

```
pessoas = {
    "nome": "joão",
    "idade": 18,
    "habilidades": ["python", "java", "php"]
}

pessoas["idade"] = 40

nome = pessoas["nome"]
idade = pessoas["idade"]
habilidades = pessoas["habilidades"]
```

Ferramentas e ambiente

- **Dicionários**
  - Outra estrutura bastante utilizada em Python são os **dicionários**.
  - Nele, você deve especificar um conjunto de **chaves** e os seus **valores**:

```python
pessoas = {
    "nome": "joão",
    "idade": 18,
    "habilidades": ["python", "java", "php"]
}

pessoas["idade"] = 40

nome = pessoas["nome"]
idade = pessoas["idade"]
habilidades = pessoas["habilidades"]
```

Nome	Tipo	Tamanho	Valor
habilidades	list	3	['python', 'java', 'php']
idade	int	1	40
nome	str	1	joão
pessoas	dict	3	{'nome':'joão', 'idade':40, 'habilidades': ['python', 'java', 'php']}

Ferramentas e ambiente

- **Operadores aritméticos**
  - Python oferece os seguintes operadores aritméticos:

Operação	Operador
Adição	+
Subtração	-
Multiplicação	*
Divisão	/
Potenciação	**
Divisão de inteiros	//
Resto da divisão	%

Ferramentas e ambiente

- **Operadores aritméticos**
  - Exemplo:

```
conta =  (9 - ( 1 + 2)) / 3.0
divisaoInteira = 10//9
outraConta = 10**2 + conta
```

Ferramentas e ambiente

- **Operadores aritméticos**
  - Exemplo:

```
conta =  (9 - ( 1 + 2)) / 3.0
divisaoInteira = 10//9
outraConta = 10**2 + conta
```

Nome	Tipo	Tamanho	Valor
conta	float	1	2.0
divisaoInteira	int	1	1
outraConta	float	1	102.0

Ferramentas e ambiente

- **Operadores relacionais**
  - Python oferece os seguintes operadores relacionais:

Operação	Operador
Maior que	>
Menor que	<
Igual a	==
Maior ou igual a	>=
Menor ou igual a	<=

Ferramentas e ambiente

- **Operadores relacionais**
  - Exemplo:

```
teste = 1 > 2
teste2 = "a" > "b"
teste3 = 2**2 == 2*2
```

Ferramentas e ambiente

- **Operadores relacionais**
  - Exemplo:

```
teste = 1 > 2
teste2 = "a" > "b"
teste3 = 2**2 == 2*2
```

Nome	Tipo	Tamanho	Valor
teste	bool	1	False
teste2	bool	1	False
teste3	bool	1	True

Ferramentas e ambiente

- **Operadores lógicos**
  - O tipo de dado Booleano, em Python, pode assumir um valor dentre 2 valores constantes: **True** ou **False**.
  - Python oferece os seguintes operadores lógicos:

Operação	Operador
e	and
ou	or
Negação	not

Ferramentas e ambiente

- **Operadores lógicos**
  - Exemplo:

```
reprovado = True and False
aprovado = True or False
naoAprovado = not aprovado
```

Ferramentas e ambiente

- **Operadores lógicos**
  - Exemplo:

```
reprovado = True and False
aprovado = True or False
naoAprovado = not aprovado
```

Nome	Tipo	Tamanho	Valor
aprovado	bool	1	True
naoAprovado	bool	1	False
reprovado	bool	1	False

Ferramentas e ambiente

- **Entrada de dados**
  - Para pedir que o usuário entre com algum valor, basta usar a função “input”.
  - Exemplo:

```
nome  = input("Qual é o seu nome?")
idade = input("Quantos anos você tem?")
cor = input("Qual é a sua cor favorita?")
```

Ferramentas e ambiente

- **Entrada de dados**
  - Para pedir que o usuário entre com algum valor, basta usar a função “input”.
  - Exemplo:

```
nome  = input("Qual é o seu nome?")
idade = input("Quantos anos você tem?")
cor = input("Qual é a sua cor favorita?")
```

Nome	Tipo	Tamanho	Valor
cor	str	1	Preto
idade	str	1	32
nome	str	1	Marco

Ferramentas e ambiente

- **Saída de dados**
  - Para imprimir na tela algum valor, basta usar a função “print”.
  - Exemplo:

```
print("Seu nome é %s, você tem %s anos e sua cor favorita é %s." %(nome, idade, cor))
```

```
Seu nome é Marco, você tem 32 anos e sua cor favorita é Preto.
```

Nome	Tipo	Tamanho	Valor
cor	str	1	Preto
idade	str	1	32
nome	str	1	Marco

Ferramentas e ambiente

- **Conversão de tipos**
  - O Python fornece algumas funções para conversão de tipos.

➢ Para string:
```
cpf = str(33508715426)
```
➢ Para um inteiro:
```
idade = int(input("Quantos anos você tem?"))
```
➢ Para um float:
```
salario = float("1500")
```
➢ Para lógico:
```
aprovado = bool(1)
```

Ferramentas e ambiente

- **Conversão de tipos**
  - O Python fornece algumas funções para conversão de tipos.

➢ Para string: `cpf = str(33508715426)`

➢ Para um inteiro: `idade = int(input("Quantos anos você tem?"))`

➢ Para um float: `salario = float("1500")`

➢ Para lógico: `aprovado = bool(1)`

Nome	Tipo	Tamanho	Valor
aprovado	bool	1	True
cpf	str	1	33508715426
idade	int	1	32
salario	float	1	1500.0

Ferramentas e ambiente

- **Manipulação de Strings**
  - String é um tipo comum e que pode ser manipulado de diversas formas:

```
string0 = 'Marco'
string1 = string0*2
string2 = string0 + ' Kappel'
teste = 'pp' in string2
string3 = string1[1]
string4 = string2[-1]
string5 = string1[0:5]

string6 = string1.upper()
string7 = string1.lower()
conta = string2.count('p')
string8 = string2.replace('Kappel','Andre')
string9 = string2.strip()
```

Ferramentas e ambiente

- **Manipulação de Strings**
  - String é um tipo comum e que pode ser manipulado de diversas formas:

```
string0 = 'Marco'
string1 = string0*2
string2 = string0 + ' Kappel'
teste = 'pp' in string2
string3 = string1[1]
string4 = string2[-1]
string5 = string1[0:5]

string6 = string1.upper()
string7 = string1.lower()
conta = string2.count('p')
string8 = string2.replace('Kappel','Andre')
string9 = string2.strip()
```

Nome	Tipo	Tamanho	Valor
conta	int	1	2
string0	str	1	Marco
string1	str	1	MarcoMarco
string2	str	1	Marco Kappel
string3	str	1	a
string4	str	1	l
string5	str	1	Marco
string6	str	1	MARCOMARCO
string7	str	1	marcomarco
string8	str	1	Marco Andre
string9	str	1	Marco Kappel
teste	bool	1	True

Ferramentas e ambiente

- **Estruturas de decisão**
  - Todas os blocos em Python são separados de acordo com a **indentação**.

```
i = 7
if i == 7:
    print("sete")
    print("sim, é igual a sete")
elif i == 8:
    print("oito")
else:
    print("Não é sete nem oito!")
    print("Que pena")
```

Ferramentas e ambiente

- **Estruturas de repetição – for**
  - O for é usado quando se quer executar um bloco de código **repetidas** vezes.

```
lista = [1, 2, 3, 4, 5]

# for
for item in lista:
    print(item)
```

Ferramentas e ambiente

- **Estruturas de repetição – for**
  - Podemos usar a função “range” para definir um **valor inicial**, um **valor de parada** e o **tamanho dos passos**:

```
# usando range
for num in range(0,10,2):
    print(num)
```

Valor de Parada

Valor Inicial

Tamanho do passo

  - Obs: o valor de parada e o tamanho do passo são opcionais!

Ferramentas e ambiente

- **Estruturas de repetição – for**
  - O for também pode ser usado para percorrer registros em um **dicionário**:

```
# for com dict
notas = {
    'Potuguês': 7,
    'Matemática': 9,
    'Lógica': 7,
    'Algoritmos': 7
}

for chave, valor in notas.items():
    print(chave,":", valor)
```

Ferramentas e ambiente

- **Estruturas de repetição – while**
  - O while é usado quando queremos que um bloco de código seja repetido até que uma **condição** seja satisfeita.

```
# while
contador = 0
while contador < 10:
    print("Valor do contador é: ",contador)
    contador += 1
```

Ferramentas e ambiente

- **Funções**
  - Todas os blocos em Python são separados de acordo com a indentação.

```
def minhaFuncao(arg1, arg2):
    # código da função
    return 1

def soma(x, y, z=0):
    soma = x + y + z
    return soma

a = soma(1,3,5)
b = soma(1,3,z=5)
c = soma(1,3)
d = soma(y=1,x=3)
```

Ferramentas e ambiente

- **Funções**
  - Todas os blocos em Python são separados de acordo com a indentação.

```python
def minhaFuncao(arg1, arg2):
    # código da função
    return 1

def soma(x, y, z=0):
    soma = x + y + z
    return soma

a = soma(1,3,5)
b = soma(1,3,z=5)
c = soma(1,3)
d = soma(y=1,x=3)
```

Nome	Tipo	Tamanho	Valor
a	int	1	9
b	int	1	9
c	int	1	4
d	int	1	4

Ferramentas e ambiente

- **Classes**
  - Python oferece suporte completo a Orientação a Objetos.

```
class Triangle(object):

    def __init__(self, angle1, angle2, angle3):
        self.angle1 = angle1
        self.angle2 = angle2
        self.angle3 = angle3

    def sum_triangle(self):
        return self.angle1 + self.angle2 + self.angle3

    def check_angles(self):
        if(self.triangle() == 180):
            return True
        else:
            return False

figure = Triangle(30, 40, 50)
print(figure.sum_triangle())
```

Ferramentas e ambiente

- **Bibliotecas**
  - Importar:

```
import numpy
```

  - Importar e definir um identificador para a biblioteca:

```
import numpy as np
```

  - Importação seletiva:

```
from math import pi
```

pandas
Data analysis

learn
Machine learning

NumPy
Scientific computing

matplotlib
2D plotting

Ferramentas e ambiente

- **Numpy arrays**
  - É uma das principais estruturas de dados utilizadas em aplicações de Machine Learning com Python.

```
minhaLista = [1,2,3,4]
meuArray = np.array(minhaLista)
meuArray2D = np.array([[1,2,3],[4,5,6]])
indice = meuArray[1]
subArray = meuArray[0:2]
todasLinhas = meuArray2D[:,0]
teste = meuArray > 3
arrayDobrado = meuArray*2
somaVetorial = meuArray + np.array([5,6,7,8])
dimensao = meuArray.shape
meuArray = np.append(meuArray,5)
meuArray = np.insert(meuArray, 1, 3)
meuArray = np.delete(meuArray,0)
media = np.mean(meuArray)
desvio = np.std(meuArray)
```

Ferramentas e ambiente

- **Numpy arrays**
  - É uma das principais estruturas de dados utilizadas em aplicações de Machine Learning com Python.

```
minhaLista = [1,2,3,4]
meuArray = np.array(minhaLista)
meuArray2D = np.array([[1,2,3],[4,5,6]])
indice = meuArray[1]
subArray = meuArray[0:2]
todasLinhas = meuArray2D[:,0]
teste = meuArray > 3
arrayDobrado = meuArray*2
somaVetorial = meuArray + np.array([5,6,7,8])
dimensao = meuArray.shape
meuArray = np.append(meuArray,5)
meuArray = np.insert(meuArray, 1, 3)
meuArray = np.delete(meuArray,0)
media = np.mean(meuArray)
desvio = np.std(meuArray)
```

Nome	Tipo	Tamanho	Valor
arrayDobrado	int32	(4,)	[2 4 6 8]
desvio	float64	1	1.019803902718557
dimensao	tuple	1	(4)
indice	int32	1	2
media	float64	1	3.4
meuArray	int32	(5,)	[3 2 3 4 5]
meuArray2D	int32	(2, 3)	[[1 2 3] [4 5 6]]
minhaLista	list	4	[1, 2, 3, 4]
somaVetorial	int32	(4,)	[6 8 10 12]
subArray	int32	(2,)	[1 2]
teste	bool	(4,)	[False False False True]
todasLinhas	int32	(2,)	[1 4]

Ferramentas e ambiente

- **Pandas**
  - Pandas é uma biblioteca de software para manipulação e análise de dados.
  - Ela oferece estruturas e operações para manipular tabelas numéricas e séries temporais.
  - Existem dois tipos principais de **estruturas de dados** no Pandas:
    - **Series**: funciona como um array unidimensional com um índice (index) que dá um rótulo a cada elemento da lista.
    - **Dataframe**: estrutura bidimensional de dados, como uma planilha, podendo ter variados tipos de dados.

- **Series**

  – Para começar a usar o Pandas, é necessário fazer a importação:

```
import pandas as pd

notas = pd.Series([2,7,5,10,6])
a = notas.values
b = notas.index
notasTurma = pd.Series([2,7,5,10,6], index=["João", "Maria", "Pedro", "Jennifer", "Luís"])
c = notasTurma["Jennifer"]
media = notasTurma.mean()
desvioPadrao = notasTurma.std()
resumo = notasTurma.describe()
```

- **Series**

  – Para começar a usar o Pandas, é necessário fazer a importação:

```
import pandas as pd

notas = pd.Series([2,7,5,10,6])
a = notas.values
b = notas.index
notasTurma = pd.Series([2,7,5,10,6], index=["João", "Maria", "Pedro", "Jennifer", "Luís"])
c = notasTurma["Jennifer"]
media = notasTurma.mean()
desvioPadrao = notasTurma.std()
resumo = notasTurma.describe()
```

Nome	Tipo	Tamanho	Valor
a	Array of int64	(5,)	[2 7 5 10 6]
b	RangeIndex	(5,)	RangeIndex: 5 entries, 0 to 4
c	int64	1	10
desvioPadrao	float	1	2.9154759474226504
media	float	1	6.0
notas	Series	(5,)	Series object of pandas.core.series module
notasTurma	Series	(5,)	Series object of pandas.core.series module
resumo	Series	(8,)	Series object of pandas.core.series module

- **Dataframes**
  - Normalmente, os dados que usaremos estarão neste formato.

```
df = pd.DataFrame({'Aluno' : ["João", "Maria", "Pedro", "Jennifer", "Luís"],
                   'Faltas' : [3,4,2,1,4],
                   'Prova' : [2,7,5,10,6],
                   'Seminário': [8.5,7.5,9.0,7.5,8.0]},
                  index=["a", "b", "c", "d", "e"])
resumo = df.describe()
coluna3porRotulo = df.loc["d"]   # Selecionar linha pelo rótulo (index)
coluna3porId = df.iloc[3]   # Selecionar linha pela posição
acima8 = df[df["Seminário"] > 8.0] # Selecionar linhas por condição
```

- **Dataframes**

  – Normalmente, os dados que usaremos estarão neste formato.

```
df = pd.DataFrame({'Aluno' : ["João", "Maria", "Pedro", "Jennifer", "Luís"],
                   'Faltas' : [3,4,2,1,4],
                   'Prova' : [2,7,5,10,6],
                   'Seminário': [8.5,7.5,9.0,7.5,8.0]},
                  index=["a", "b", "c", "d", "e"])
resumo = df.describe()
coluna3porRotulo = df.loc["d"]   # Selecionar linha pelo rótulo (index)
coluna3porId = df.iloc[3]   # Selecionar linha pela posição
acima8 = df[df["Seminário"] > 8.0] # Selecionar linhas por condição
```

Nome	Tipo	Tamanho	Valor
acima8	DataFrame	(2, 4)	Column names: Aluno, Faltas, Prova, Seminário
coluna3porId	Series	(4,)	Series object of pandas.core.series module
coluna3porRotulo	Series	(4,)	Series object of pandas.core.series module
df	DataFrame	(5, 4)	Column names: Aluno, Faltas, Prova, Seminário
resumo	DataFrame	(8, 3)	Column names: Faltas, Prova, Seminário

FIM